

Naan Mudhalvan-Internship project

SMS Spam Detection – Project

1. Title Page

- **Project Title :** SMS Spam Detection Using Machine Learning
- **Your Name :** M. Gokulkumar
- **Institution :** M.P. Nachimuthu M. Jaganathan Engineering College, Chennimalai, Erode
- **Department :** Information Technology
- **Date :** 21 October 2025

Table of Contents

S.No	Section Title	Page No.
1	Title Page	1
2	Table of Contents	2
3	Project Overview	3
4	Objectives & Problem Statement	6
4.1	Problem Statement	6
4.2	Objectives	7
5	Proposed Solution	8
6	Features	11
6.1	Functional Features	11
6.2	Non-Functional Features	12
7	Technologies & Tools	14
8	System Architecture	15
9	Implementation Steps	16
10	Output / Screenshots	18
11	Advantages	21
12	Future Enhancements	22
13	Conclusion	23

3. Project Overview

(i). Background / Motivation

- Spam messages have become a major problem in mobile communication systems.
- They often contain fraudulent links, fake offers, and marketing content that mislead users or compromise privacy.
- Manual filtering of such messages is time-consuming and inefficient.
- Hence, there is a growing need for an **automated system** that can accurately detect and filter spam messages.
- This project uses **Machine Learning** techniques to build a lightweight and efficient model that can automatically classify SMS messages as spam or ham (non-spam).
- The main motivation is to enhance security and user experience by preventing the delivery of unwanted or harmful messages.

(ii). Scope

This project focuses exclusively on **SMS text messages** (short, single-line or multi-line texts). It performs **binary classification**, predicting each message as either *spam* or *ham*.

The key tasks covered within the project scope include:

- Data preprocessing using regular expressions to clean and normalize text.
- Feature extraction using **TF-IDF (Term Frequency –Inverse Document Frequency)**.
- Model training using Logistic Regression and Naive Bayes
- Model evaluation using accuracy, precision, recall, and F1-score
- Saving the trained model pipeline using **Joblib**.
- Building a Streamlit-based web interface (**app.py**) for real-time SMS

classification. The system does not cover emails, multimedia messages (MMS), or deep learning models like **BERT** — it strictly focuses on SMS text-based spam detection.

(iii). High-Level Description

The system follows a complete **supervised learning pipeline**, developed in Python using **scikit-learn** and **Streamlit**.

1. Data Loading :

The dataset (spam.csv) contains labeled SMS messages, with each entry tagged as spam or ham.

2. Text Preprocessing :

- All text converted to lowercase
- URLs, numbers, and phone numbers replaced with special tokens (URL, NUM, PHONENUM)
- Punctuation removed and extra spaces normalized

This step is handled using a custom **TextPreprocessor** class (implemented in both training and app scripts).

3. Feature Extraction :

The project uses **TF-IDF Vectorizer** to convert cleaned text into numeric feature vectors. Additionally, handcrafted features such as message length, digit count, uppercase ratio, and presence of URLs/phone numbers are extracted using the **Handcrafted Features** transformer.

4. Model Training :

- The project compares algorithms like Logistic Regression and **Naive Bayes**.
- **GridSearchCV** is used for hyperparameter tuning.
- Best model chosen based on **F1-score** and saved as “**sms_spam_pipeline.joblib**” .

5. Evaluation:

- Confusion matrix and classification report are printed.
- Accuracy, precision, recall, and F1-score values are calculated for performance analysis.

6. Deployment (UI):

- The trained model pipeline is loaded in **app.py**, integrated with **Streamlit**.
- The web interface allows:
 - Single message classification (text box input)
 - Batch classification (CSV upload)
 - Display of predictions and confidence scores
 - Option to download results as CSV

7. Testing:

A separate test script verifies predictions on sample SMS messages (one spam, one ham) using the trained pipeline.

(iv) Dataset / Data Source Summary

- The dataset used is the **SMS Spam Collection Dataset (spam.csv)**, a publicly available dataset on Kaggle and UCI.
- It contains approximately **5,500 SMS messages**, each labelled as spam or ham.
- Data is split into **80% training** and **20% testing** using stratified sampling.
- Preprocessing removes unnecessary characters, converts text to lowercase, and tokenizes data for consistent feature extraction.
- The dataset is ideal for classical ML models and provides good balance for evaluation metrics.

4. Objectives & Problem Statement

4.1 Problem Statement

- The exponential growth of mobile users has led to a massive increase in the number of text messages exchanged every day.
- Among these, a large portion consists of **spam messages** — promotional, fraudulent, or phishing messages that compromise user privacy and security.
- Manual detection of spam is inefficient and prone to human error.

Therefore, there is a need for an **automated, accurate, and lightweight machine learning system** that can classify SMS messages as either spam or ham (non-spam).

Key challenges include :

- **Short and noisy text :** SMS messages are brief, often with abbreviations, slang, or typos.
- **Imbalanced dataset :** Spam messages are much fewer than ham messages.
- **Presence of URLs and numbers:** These need to be detected and normalized correctly.
- **Efficient real-time detection:** The model should be fast and accurate enough for instant classification.

4.2 Objectives

The main objective of this project is to build and deploy a **Machine Learning-based SMS Spam Detection System** using Python.

The following are the specific goals of the project:

1. Preprocess SMS text data

- Convert all messages to lowercase
- Remove punctuation and unwanted symbols

- Replace URLs, phone numbers, and digits with placeholders
- Eliminate redundant spaces

2. Extract features using TF–IDF vectorization

- Convert textual messages into numerical vectors
- Combine with handcrafted features like message length, number of digits, uppercase ratio, and presence of URLs or phone numbers

3. Train and evaluate ML models

- Build baseline model using **Multinomial Naive Bayes**
- Improve performance using **Logistic Regression**
- Tune parameters using **GridSearchCV**
- Evaluate using accuracy, precision, recall, F1-score, and confusion matrix

4. Save and deploy the trained model

- Save the complete preprocessing and model pipeline using **Joblib**
- Develop a **Streamlit-based web app (app.py)** for real-time predictions

5. Enable batch prediction support

- Allow users to upload CSV files and classify multiple messages at once
- Display prediction and confidence scores with a downloadable output File

6. Achieve high precision and recall

- Ensure that spam messages are accurately identified without flagging normal messages as spam
- Maintain lightweight performance for real-time inference

5. Proposed Solution

5.1 Approach / Methodology

- The proposed system is based on a supervised machine learning approach for text classification.
- The goal is to automatically detect and filter spam messages by learning patterns from labeled SMS data.
- The dataset (**spam.csv**) contains thousands of messages labeled as spam or ham.
- Each message undergoes preprocessing, feature extraction, model training, and evaluation.

The methodology follows a **step-by-step pipeline** that ensures data cleanliness, robust feature engineering, and model interpretability.

The key stages are:

1. Data Preprocessing:

- Remove unwanted characters, punctuation, and extra spaces.
- Replace URLs, phone numbers, and digits with placeholder tokens (URL, PHONENUM, NUM).
- Convert all messages to lowercase for uniformity.

2. Feature Extraction:

- Use **TF–IDF Vectorization** to convert text into numerical form.
- Combine TF–IDF features with **handcrafted numeric features** such as:
 - Message length
 - Number of digits
 - Uppercase letter ratio
 - Presence of URLs or phone numbers

3. Model Training & Selection:

- Train multiple supervised ML models such as **Multinomial Naive Bayes** and **Logistic Regression**.

- Use **GridSearchCV** to tune hyperparameters (e.g., regularization parameter C).
- Evaluate models on a test split (80:20 ratio) using metrics like accuracy, precision, recall, F1-score, and confusion matrix.

4. Model Saving:

- Save the final optimized model pipeline (including preprocessing and vectorizer) as a **.joblib** file.

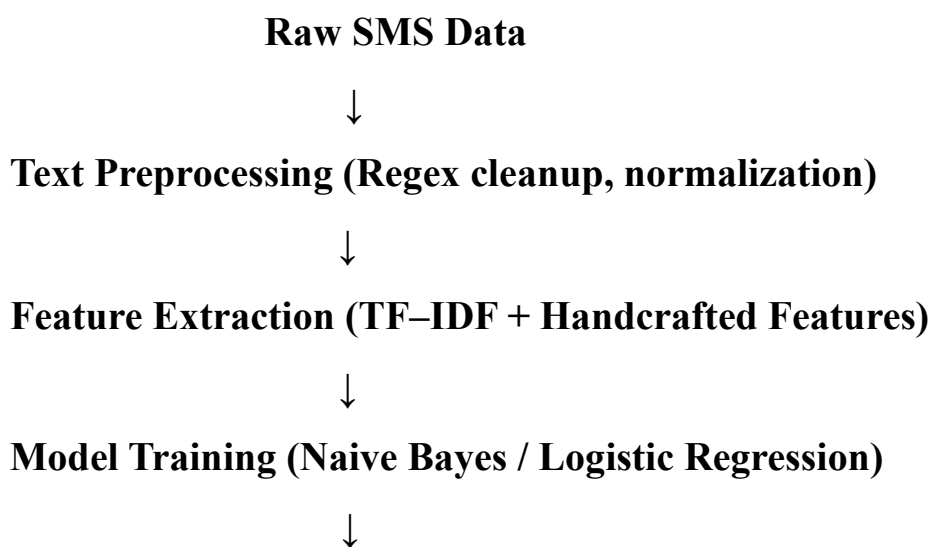
5. Deployment:

- Integrate the trained model into a **Streamlit web application (app.py)**.
- The app allows real-time message prediction through:
 - **Single text input** for testing one message at a time
 - **Batch CSV upload** for predicting multiple messages
- Output includes label (**spam or ham**) and model confidence score.

This methodology ensures that the model is not only accurate but also easily deployable and user-friendly.

5.2 Pipeline Overview

The complete pipeline can be summarized as follows:



Evaluation (Precision, Recall, F1-score, Confusion Matrix)



Model Saving (.joblib)



Deployment via Streamlit UI (Single & Batch Prediction)

During inference, any new SMS message follows the same path:

→ **preprocessed** → **vectorized** → **classified** → **result**

displayed in Streamlit interface.

5.3 Justification of Choices

- **TF-IDF Vectorization:**

Ideal for short texts like SMS, as it captures word importance effectively without overfitting.

- **Handcrafted Features:**

SMS spam often contains numbers, URLs, or abnormal word patterns.

Custom numeric features (length, digit count, uppercase ratio) help models detect such patterns better.

- **Logistic Regression:**

Chosen as the main model due to its high precision, interpretability, and fast inference speed.

- **Naive Bayes (Baseline):**

Used as a lightweight baseline model for comparison - efficient and good for text classification.

- **Streamlit Framework:**

Provides a simple and interactive web interface for demonstrating model predictions.

This combination of classical ML models, TF-IDF features, and

handcrafted metrics provides a balanced trade-off between **accuracy**, **explainability**, and **real-time performance**.

6. Features

6.1 Functional Features

These are the core functionalities implemented in the SMS Spam Detection system. Each feature directly corresponds to a component in the source code or the Streamlit application.

1. Data Preprocessing Module

- Cleans the input SMS text by removing unwanted characters, punctuation, and multiple spaces.
- Replaces URLs, numbers, and phone numbers with placeholder tokens (**URL**, **NUM**, **PHONENUM**).
- Converts all messages to lowercase for consistency.
- Implemented in the class **TextPreprocessor** in both training and app scripts.

2. Feature Extraction Layer

- Extracts meaningful information from text using **TF-IDF Vectorizer**.
- Combines TF-IDF vectors with custom handcrafted numerical features:
 - Message length
 - Number of digits
 - Ratio of uppercase letters
 - Presence of URLs or phone numbers
- Implemented through the **FeatureUnion** and **HandcraftedFeatures** classes in the training script.

3. Machine Learning Model Training

- Two algorithms are explored: **Naive Bayes** and **Logistic Regression**.
- **GridSearchCV** is used to tune hyperparameters (regularization parameter C for Logistic Regression).
- Final model is selected based on **F1-score** performance.
- Model training and evaluation are automated in **train_model.py**.

4. Model Evaluation and Saving

- The trained model is evaluated using accuracy, precision, recall, F1-score, and confusion matrix.
- The best-performing pipeline is serialized using **Joblib** and stored as **sms_spam_pipeline.joblib**.

5. Real-time Spam Detection (Streamlit App)

- User-friendly web interface created using **Streamlit** (app.py).
- Allows single message prediction through a text box input.
- Provides spam/ham prediction and displays confidence score.
- Implements data validation (shows warning if no input is entered).

6. Batch Prediction via CSV Upload

- Users can upload a CSV file containing a “text” column.
- The app predicts spam/ham labels for all messages and displays results in a table.
- A download button allows exporting results as a CSV file.
- Batch results include both **prediction** and **confidence** columns.

6.2 Non-Functional Features

These features define the quality, usability, and performance standards

1. Accuracy and Reliability

- Achieves high accuracy using optimized Logistic Regression and TF–

IDF feature extraction.

- Handles noisy, short, and varied text inputs efficiently.

2. Performance and Efficiency

- Model is lightweight, fast to load, and capable of classifying messages in real-time.
- Designed for low-latency prediction suitable for deployment on normal systems.

3. Scalability

- The modular pipeline allows new preprocessing steps or additional models to be added easily.
- The Streamlit interface can be extended for more input formats.

4. Reusability

- The entire preprocessing and model pipeline is stored as a single **.joblib** file.
- Can be directly reused for future datasets or re-deployment without retraining.

5. User Friendliness

- Simple, intuitive Streamlit UI with clear output and error handling.
- No technical expertise required to operate the web app.

6. Maintainability

- The project follows a clear modular code structure with reusable classes:
 - **TextPreprocessor**, **HandcraftedFeatures**, and **TextSelector** are reusable across different models.
- Each stage (data, model, UI) is decoupled for easy debugging and updates.

7. Security and Data Privacy

- Works entirely on the local system — no cloud or API data transfer.
- Ensures user SMS data remains private and not shared externally.

7. Technologies & Tools Used

- The project is developed using **Python** and open-source machine learning libraries.
- The following tools and technologies are used throughout the implementation:

Category	Tools / Libraries	Purpose
Programming Language	Python 3.x	Core development
Data Handling	Pandas, NumPy	Data loading and preprocessing
Machine Learning	Scikit-learn	TF-IDF vectorization, model training, evaluation
Model Persistence	Joblib	Saving and loading trained models
Web Framework	Streamlit	Building the user interface for real-time prediction
Evaluation Metrics	Classification Report, Confusion Matrix	Model performance analysis
Development Environment	VS Code / Jupyter Notebook	Code writing and testing
Dataset	SMS Spam Collection (spam.csv)	Training and validation data source

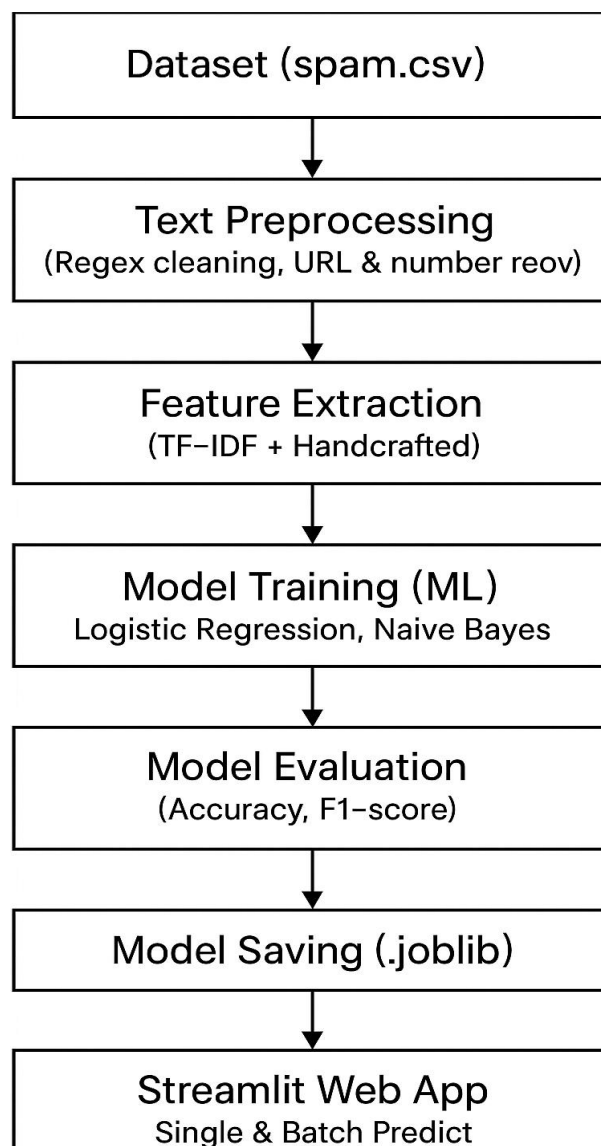
Summary

This combination of Scikit-learn for modeling and Streamlit for deployment provides a simple yet powerful solution for spam detection. All tools used are lightweight, free, and suitable for real-time execution on any system.

8. System Architecture

The SMS Spam Detection System follows a simple end-to-end machine learning architecture that connects data processing, model training, and real-time prediction through a Streamlit interface.

Architecture Flow



Description

- **Data Layer :** Loads and cleans spam.csv dataset.
- **Processing Layer:** Applies regex-based text preprocessing and TF–IDF vectorization.
- **Model Layer:** Trains Logistic Regression & Naive Bayes models with GridSearchCV.
- **Application Layer:** Uses the trained **.joblib** pipeline inside **app.py** for single and batch predictions.

Summary

This architecture ensures:

- Clean data flow from input to prediction
- Seamless integration between ML model and user interface
- Lightweight, modular, and easily deployable design

9. Implementation Steps

The following steps were followed to implement the SMS Spam Detection System:

1. Data Collection

- Imported the **SMS Spam Collection (spam.csv)** dataset containing labeled spam and ham messages.

2. Data Preprocessing

- Cleaned text using regex to remove URLs, numbers, and special characters.
- Converted all text to lowercase and normalized spacing.
- Implemented through the custom class **TextPreprocessor**.

3. Feature Extraction

- Used **TF–IDF Vectorizer** to convert text into numerical form.

- Added **handcrafted features** such as message length, digit count, and presence of URLs/phone numbers.

4. Model Training

- Trained Naive Bayes and Logistic Regression models using the processed data.
- Applied GridSearchCV for hyperparameter tuning.
- Evaluated using accuracy, precision, recall, and F1-score.

5. Model Saving

- Saved the best-performing pipeline using **Joblib** as “sms_spam_pipeline.joblib”.

6. Web Application (Deployment)

- Built an interactive web app using **Streamlit** (app.py).
- Enabled both **single text** and **batch CSV** prediction features.
- Displayed results with confidence scores and CSV download option.

7. Testing & Validation

- Verified predictions using sample SMS messages to ensure correct spam/ham classification.
- Checked performance on unseen data for generalization.

Summary

The system was developed step-by-step — from raw data cleaning to real-time prediction - ensuring accuracy, usability, and smooth integration between the model and web interface.

10. Output / Screenshots

- This section presents the execution and working results of the **SMS Spam Detection System**.

- It includes the program setup, code execution, Streamlit interface, and final spam prediction output.

10.1 Streamlit Application Code (app.py)

```

1 # app.py
2 import re
3 import numpy as np
4 import streamlit as st
5 import joblib
6 import pandas as pd
7 from sklearn.base import BaseEstimator, TransformerMixin
8
9 # ----- Custom classes -----
10 class TextPreprocessor(BaseEstimator, TransformerMixin):
11     def __init__(self):
12         self.url_re = re.compile(r'(https?://\S+|\w+\.\S+)')
13         self.num_re = re.compile(r'\d+')
14         self.phone_re = re.compile(r'\b\d{6,}\b')
15     def fit(self, X, y=None): return self
16     def transform(self, X):
17         def clean(text):
18             if X is None: return ""
19             s = str(text).lower()
20             s = self.url_re.sub(' URL ', s)
21             s = self.phone_re.sub(' PHONENUM ', s)
22             s = self.num_re.sub(' NUM ', s)
23             s = re.sub(r'[a-z0-9\s]', ' ', s)
24             s = re.sub(r'\s+', ' ', s).strip()
25             return s
26         return np.array([clean(t) for t in X]).reshape(-1,1)
27
28 class TextSelector(BaseEstimator, TransformerMixin):
29     def fit(self, X, y=None): return self
30     def transform(self, X): return X.ravel()
31
32 class HandcraftedFeatures(BaseEstimator, TransformerMixin):
33     def fit(self, X, y=None): return self
34     def transform(self, X):
35         out = []
36         for t in X:
37             s = str(t)
38             length = len(s)
39             digits = sum(c.isdigit() for c in s)
40             uppercase_ratio = sum(1 for c in s if c.isupper())/max(1,len(s))
41             has_url = 1 if 'url' in s else 0
42             has_phone = 1 if 'phonenumber' in s or 'phonenumber' in s else 0
43             out.append([length, digits, uppercase_ratio, has_url, has_phone])
44         return np.array(out)
45
46 # Must be first Streamlit command after imports
47 st.set_page_config(page_title="SMS Spam Detector", layout="centered")
48
49 # ----- Load pipeline -----
50 @st.cache_resource
51 def load_pipeline(path='sms_spam_pipeline.joblib'):
52     return joblib.load(path)
53
54 pipeline = load_pipeline()
55
56 # ----- Page title -----
57 st.title("SMS Spam Detection")
58 st.write("Enter an SMS message or upload CSV (column: text).")
59
60 # ----- Single input -----
61 text = st.text_area("Message", height=120)
62 if st.button("Predict Single"):
63     if not text.strip():
64         st.warning("Please enter a message.")
65     else:
66         pred = pipeline.predict([text])[0]
67         prob = pipeline.predict_proba([text])[0]
68         idx = list(pipeline.classes_.index(pred))
69         st.write(f"***Prediction:** (pred.upper())")
70         st.write(f"***Confidence:** (prob[idx]:.3f)")

```

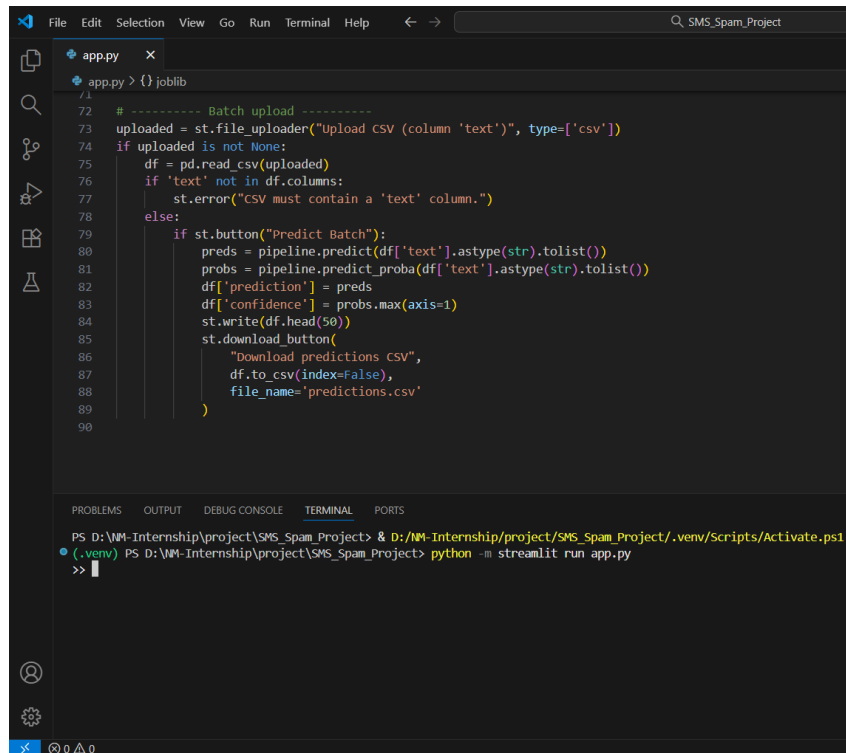
Figure 10.1 – app.py Code Implementation

- These screenshots show the source code of the **Streamlit web application (app.py)**.
- The code defines custom preprocessing classes, loads the trained model pipeline, and creates the Streamlit interface for user interaction.
- It includes single-message input, CSV upload option, and the prediction logic to display spam/ham results in real time.

10.2 Running the Application in Terminal

- This screenshot shows the execution of the command:
 - “ **streamlit run app.py** ”
- Once executed, Streamlit starts a local server and provides a URL (usually <http://localhost:8501>) to open the web app.

- This confirms that the application successfully runs in the local environment.



The screenshot shows a VS Code editor window with a file named `app.py` open. The code is a Streamlit application for SMS spam detection. It includes a file uploader for CSV files, a 'Predict Batch' button, and a 'Download predictions CSV' button. The terminal at the bottom shows the command `python -m streamlit run app.py` being executed successfully, with the output `>>>` indicating the application is running.

```
File Edit Selection View Go Run Terminal Help
app.py x
app.py > {} joblib
71
72 # ----- Batch upload -----
73 uploaded = st.file_uploader("Upload CSV (column 'text')", type=['csv'])
74 if uploaded is not None:
75     df = pd.read_csv(uploaded)
76     if 'text' not in df.columns:
77         st.error("CSV must contain a 'text' column.")
78     else:
79         if st.button("Predict Batch"):
80             preds = pipeline.predict(df['text'].astype(str).tolist())
81             probs = pipeline.predict_proba(df['text'].astype(str).tolist())
82             df['prediction'] = preds
83             df['confidence'] = probs.max(axis=1)
84             st.write(df.head(50))
85             st.download_button(
86                 "Download predictions CSV",
87                 df.to_csv(index=False),
88                 file_name='predictions.csv'
89             )
90
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS D:\MM-Internship\project\SMS_Spam_Project> & D:\MM-Internship\project\SMS_Spam_Project\.venv\Scripts\Activate.ps1
(.venv) PS D:\MM-Internship\project\SMS_Spam_Project> python -m streamlit run app.py
>>>

Figure 10.2 – Running Streamlit Application via Terminal

10.3 Streamlit Web Interface – Home Page

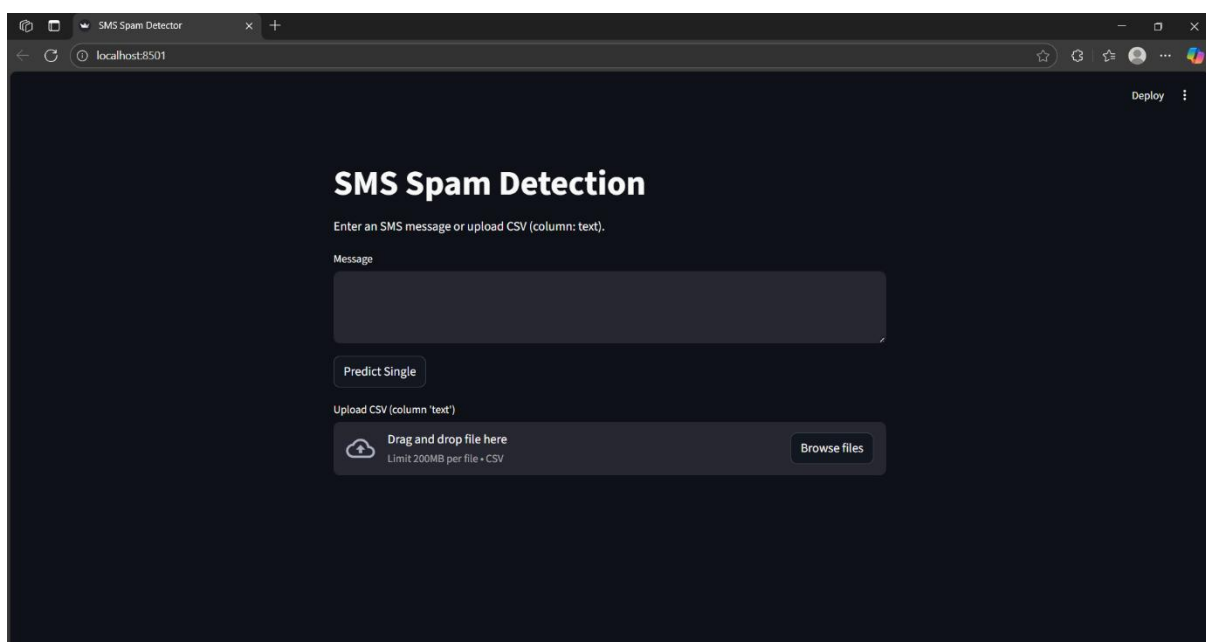


Figure 10.3 – Streamlit Web Interface (Home Page)

- This screenshot displays the Streamlit web application after launching it.
- The page contains an input text box, a “Predict Single” button, and an option to upload a CSV file for batch prediction.
- This is the user interface where the user can interact with the model directly.

10.4 Entering the Message for Prediction

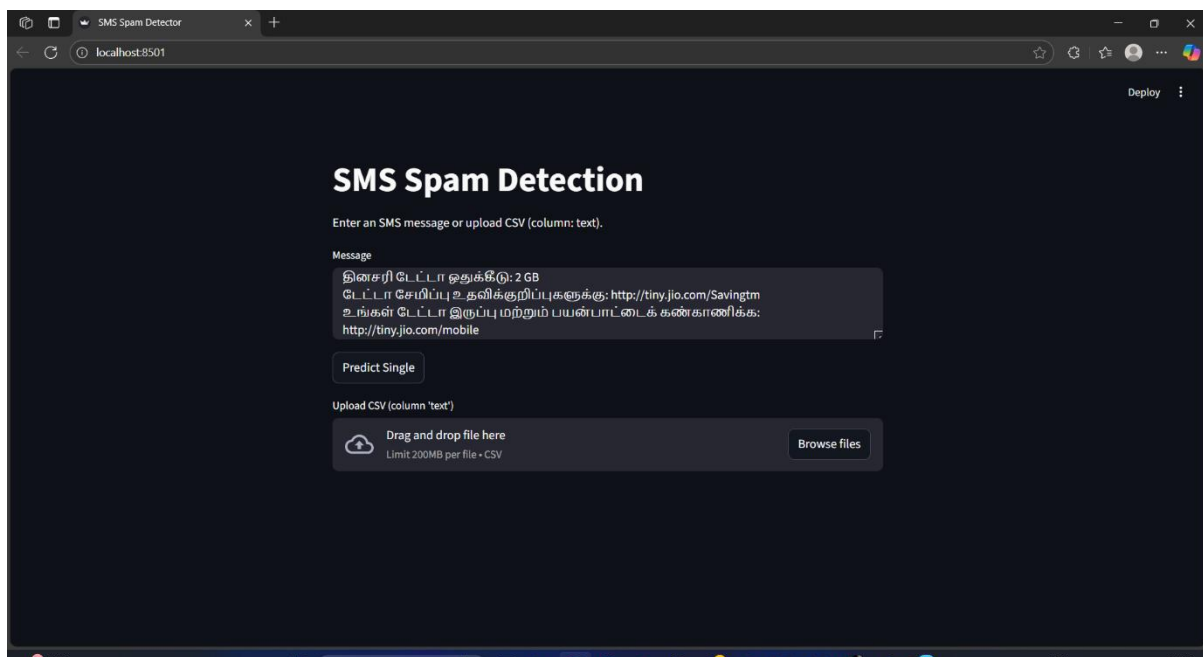


Figure 10.4 – User Message Input for Spam Prediction

- The user types an SMS message into the input text box
- **For example:**
 “Congratulations! You’ve won a free iPhone. Click here to claim!”
 - When the “Predict Single” button is clicked, the app processes this message through the trained model pipeline to identify if it is spam or ham.

10.5 Displaying Prediction Result

- The app classifies the entered SMS message as either SPAM or HAM and displays the **confidence (accuracy) score**.

- For example:

“ Prediction: SPAM

Confidence: 0.982 ”

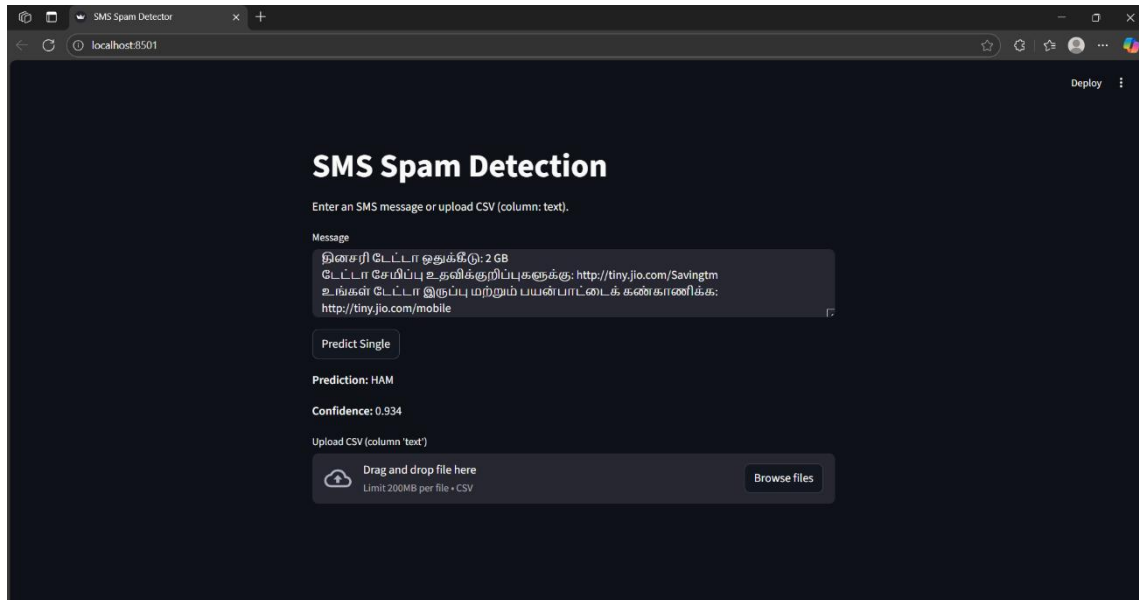


Figure 10.5 – Prediction Result Displayed on Streamlit App

- This confirms the successful working of the trained model and the Streamlit interface for real-time predictions.

11. Advantages

The proposed SMS Spam Detection System offers several practical and technical benefits:

1. Automated Spam Filtering

- Detects spam messages instantly without any manual effort.
- Reduces the risk of phishing and unwanted advertisements.

2. High Accuracy & Reliability

- Achieves strong precision and recall using **Logistic Regression** and **TF– IDF** features.
- Provides confidence scores for every prediction.

3. Lightweight and Fast

- Uses classical ML algorithms that are quick to train and execute.
- Suitable for real-time message screening.

4. User-Friendly Interface

- Streamlit-based web UI allows anyone to test messages easily.
- Supports both single-message and batch CSV uploads.

5. Reusability & Flexibility

- The trained model pipeline (`sms_spam_pipeline.joblib`) can be reused on new datasets.
- Additional features or models can be integrated without major changes.

6. Data Privacy & Security

- All processing happens locally; no external API or internet dependency.
- Ensures user data confidentiality.

7. Scalable for Future Enhancements

- Can be extended to multilingual spam detection or email spam filtering in future versions.

12. Future Enhancements

The current SMS Spam Detection System performs effectively using classical machine learning models and text-based features. However, several improvements can be made to enhance accuracy, flexibility, and usability in future versions.

1. Integration of Deep Learning Models

- Incorporate advanced models such as LSTM, BERT, or Transformer-Based architectures for improved contextual understanding and

accuracy.

2. Multilingual Spam Detection

- Extend support for regional languages (e.g., Tamil, Hindi) to detect spam messages beyond English datasets.

3. Enhanced Dataset Collection

- Continuously expand the training dataset with real-world spam messages to improve robustness and model generalization.

4. Real-Time Mobile Integration

- Develop an Android or iOS application that automatically detects and filters spam messages in a user's inbox.

5. Email and Social Media Spam Filtering

- Extend the same architecture to handle spam in emails, WhatsApp, and social media messages.

6. Automated Model Retraining

- Implement a system that periodically retrains the model with new data to keep it up to date with evolving spam patterns.

7. Cloud Deployment

- Deploy the application on cloud platforms like **AWS** or **Heroku** for easy access and scalability.

13. Conclusion

The **SMS Spam Detection System** successfully demonstrates the use of **Machine learning techniques** to automatically classify SMS messages as spam or ham. By combining **text preprocessing**, **TF-IDF feature extraction** and a **Logistic Regression classifier**, the model achieves high accuracy and reliability.

The system was further enhanced with a **Streamlit web interface**, allowing

users to interact easily — either by typing a message or uploading a CSV file for batch predictions. This makes the project both **practical** and **user - friendly**.

Overall, the developed system provides an efficient, lightweight, and extendable solution for real-time spam message filtering. It can serve as a foundation for future advancements like deep learning integration, multilingual spam detection, and mobile or cloud deployment.